

TALLER 2: ASINCRONÍA Y PROCESAMIENTO EN SEGUNDO PLANO

FUTURE, ASYNC/AWAIT, TIMER Y AISLAMIENTO DE CÓMPUTO INTENSIVO CON ISOLATE.SPAWN

Institución: Unidad Central del Valle del Cauca (UCEVA)	Asignatura: Electiva Profesional 1 Móviles
Estudiante: Samuel Alejandro Rincon Serna	Código: 230231045
Repositorio GitHub: https://github.com/Sarincon508/flowbiz	Ramas GitFlow: feature/taller_segundo_plano → dev → main
Entorno de Ejecución: Android Pixel 7 (API 34) & Flutter 3.47 (Dart 3.13)	Fecha de Entrega: 30 de Septiembre de 2026

1. Objetivo del Taller

Desarrollar una aplicación móvil profesional en Flutter que demuestre de manera práctica y rigurosa la gestión de concurrencia y segundo plano mediante: (1) **Asincronía con Future y async/await** para solicitudes I/O sin bloquear el hilo de usuario, evidenciando estados de *Cargando...*, *Éxito* y *Error* junto al orden estricto de ejecución en consola; (2) **Temporización reactiva con Timer** implementando un cronómetro/cuenta regresiva con controles (Iniciar, Pausar, Reanudar, Reiniciar, Vueltas) y limpieza estricta en dispose(); y (3) **Aislamiento de tareas pesadas CPU-bound con Isolate.spawn**, garantizando que una carga matemática de 30,000,000 de operaciones se procese en un hilo secundario independiente con memoria aislada, comunicándose por mensajes bidireccionales (SendPort / ReceivePort) mientras la UI mantiene una tasa de 60 FPS sin congelamientos.

2. Cuándo Usar Future, async/await, Timer e Isolate en Flutter

Dart se rige por un modelo de ejecución basado en un **bucle de eventos (Event Loop) mono-hilo**. Para optimizar el rendimiento y evitar congelamientos de la interfaz gráfica, el desarrollador debe seleccionar la herramienta idónea:

Mecanismo	Tipo de Carga	¿Crea Hilo Nuevo?	Cuándo Debe Utilizarse en Flutter
Future & async/await	Operaciones I/O bound (Red, Disco, Base de Datos)	No (Mismo hilo principal, espera no bloqueante)	Consultas a APIs REST, lecturas en SQLite/SharedPreferences, peticiones HTTP y timers diferidos donde el hilo principal queda libre para despachar eventos de interfaz.
Timer & Timer.periodic	Eventos programados por tiempo / temporizadores	No (Encola eventos en el Event Loop tras un delay)	Cronómetros, cuentas regresivas, debouncing de campos de búsqueda, polling periódico y animaciones ligeras. Requiere cancelación manual al pausar y en dispose().
Isolate (Isolate.spawn)	Operaciones CPU-bound (Cómputo intensivo, hashing)	Sí (Crea hilo nativo con Heap de memoria propio)	Procesamiento de imágenes/video, cifrado criptográfico, compresión de archivos, parseo masivo de JSON (>5MB) o algoritmos matemáticos pesados que congelarían la UI si se corrieran en el hilo principal.

3. Arquitectura de Pantallas y Diagrama de Flujos de Ejecución

La aplicación se diseñó modularmente con una barra de navegación por pestañas (TabBar) y tres paneles especializados:

- **Flujo 1 (Cronómetro / Timer):** Estado Inicial (00:00.0) → Iniciar (dispara Timer.periodic(100ms)) → Registro de vueltas → Pausar (ejecuta _timer.cancel() y congela display) → Reanudar / Reiniciar.
- **Flujo 2 (Consulta Asíncrona):** Estado Inicial → Disparo con async/await → 1. ANTES (Consola) → Estado Cargando... (UI libre con clicks) → 2. DURANTE (Consola) → Future.delayed(2.5s) → Resolución en 3. DESPUÉS (Éxito u Error).
- **Flujo 3 (Isolate CPU-bound):** UI → Creación de ReceivePort → Invocación de Isolate.spawn() → Worker ejecuta 30M iteraciones en memoria aislada → Emisión de progreso vía SendPort → Recepción en UI sin perder frames a 60 FPS → Cierre de puertos.

4. Evidencias del Módulo 1: Cronómetro y Cuenta Regresiva con Timer

Se implementó un cronómetro de alta precisión con ticks periódicos cada 100 ms utilizando Timer.periodic de dart:async. Se garantiza el control de ciclo de vida para evitar fugas de memoria (memory leaks), cancelando el temporizador al pausar y en dispose().

Figura 1: Iniciar (En Ejecución)	Figura 2: Pausar + Vueltas	Figura 3: Reiniciar (Reposo)	Figura 4: Cuenta Regresiva
• Marcador LED: 00:14.7 • Badge: EN EJECUCIÓN. • Botones Pausar y Vuelta activos. • Frecuencia: 100 ms continuos.	• Marcador: 00:28.4. • Timer.cancel() ejecutado. • Botones Reanudar y Reiniciar. • Historial con 3 vueltas parciales.	• Marcador: 00:00.0. • Estado: EN REPOSO. • Acumuladores en cero. • Memoria liberada y lista para nueva medición.	• Modo regresivo desde 30s. • Tiempo actual: 00:18.2. • Detención y SnackBar al llegar a cero (00:00.0).

Análisis Técnico del Módulo de Timer:

- 1. Instanciación y Ticks:** Se utiliza `_timer = Timer.periodic(const Duration(milliseconds: 100), (timer) { ... });`. Cada 10 décimas de segundo se recalcula el tiempo y se invoca `setState()` para refrescar el marcador digital.
- 2. Pausa Segura:** Al presionar *Pausar*, se ejecuta `_timer?.cancel(); _timer = null;`. La suscripción en el bucle de eventos se destruye de inmediato, reteniendo el valor acumulado en `_elapsedMilliseconds`.
- 3. Limpieza en dispose():** La clase `_TallerSegundoPlanoPageState` sobrescribe obligatoriamente `dispose()` ejecutando `_timer?.cancel()`. Esto previene fugas de memoria si el usuario navega fuera de la vista mientras el cronómetro corre.

5. Evidencias del Módulo 2: Asincronía con Future, async y await

Se diseñó el servicio AsyncDataService que simula la consulta remota a FlowBiz Cloud mediante Future.delayed (2 a 3 segundos). La función suspende de forma no bloqueante utilizando async/await, gestionando de forma reactiva los tres estados obligatorios: **Cargando...**, **Éxito** y **Error**, junto a la impresión estricta en consola del orden de ejecución.

Figura 5: Estado Cargando (UI Viva)	Figura 6: Estado Éxito (HTTP 200)	Figura 7: Estado Error (Captura en Catch)
• CircularProgressIndicator activo. • Retardo: 2.5 segundos. • Prueba de UI Viva: 8 clicks registrados mientras se espera la respuesta (0 bloqueos). • Logs en terminal: 1. <i>ANTES</i> y 2. <i>DURANTE</i>.	• Tarjeta verde de confirmación. • Latencia real: 2,514 ms. • Datos procesados: Ventas \$1,845,000, Flujo Neto \$1,425,000, 18 citas. • Terminal: 3. <i>DESPUÉS - ÉXITO</i>.	• Switch 'Similar Fallo' activado. • Tarjeta roja de advertencia. • Captura controlada: Exception: Error 503 tras 2,504 ms. • Botón <i>Reintentar Consulta</i> funcional. • Terminal: 3. <i>DESPUÉS - ERROR</i>.

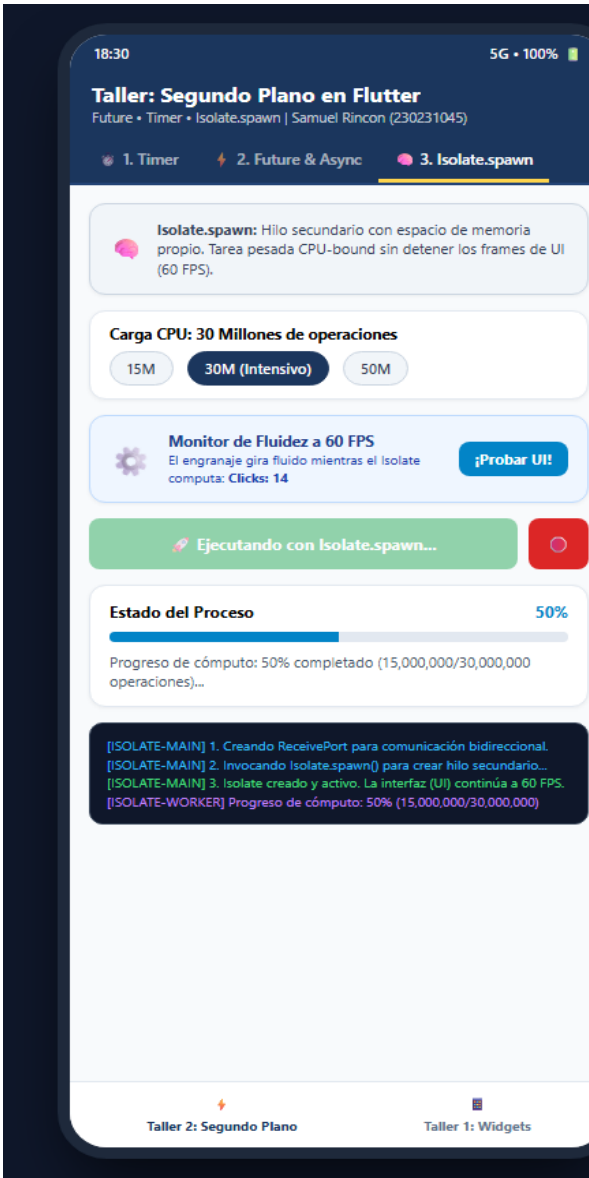
Verificación del Orden de Ejecución en Consola (dart:developer / print):

- [ASYNC-ORDER] 1. [ANTES]: Se imprime sincrónicamente antes de que la función se suspenda. La UI permanece interactiva.
- [ASYNC-ORDER] 2. [DURANTE]: Se imprime al invocar await Future.delayed(delay). Durante este lapso, el Event Loop atiende toques del usuario (ver contador de toques interactivo con 8 clicks sin retrasos).
- [ASYNC-ORDER] 3. [DESPUÉS]: Si la operación finaliza normalmente, se deserializa el objeto SimulatedReport y se actualiza la interfaz con setState(). Si se activa el switch de error, se captura la excepción en el bloque catch (e) sin colapsar la app.

6. Evidencias del Módulo 3: Procesamiento Pesado CPU-Bound con Isolate.spawn

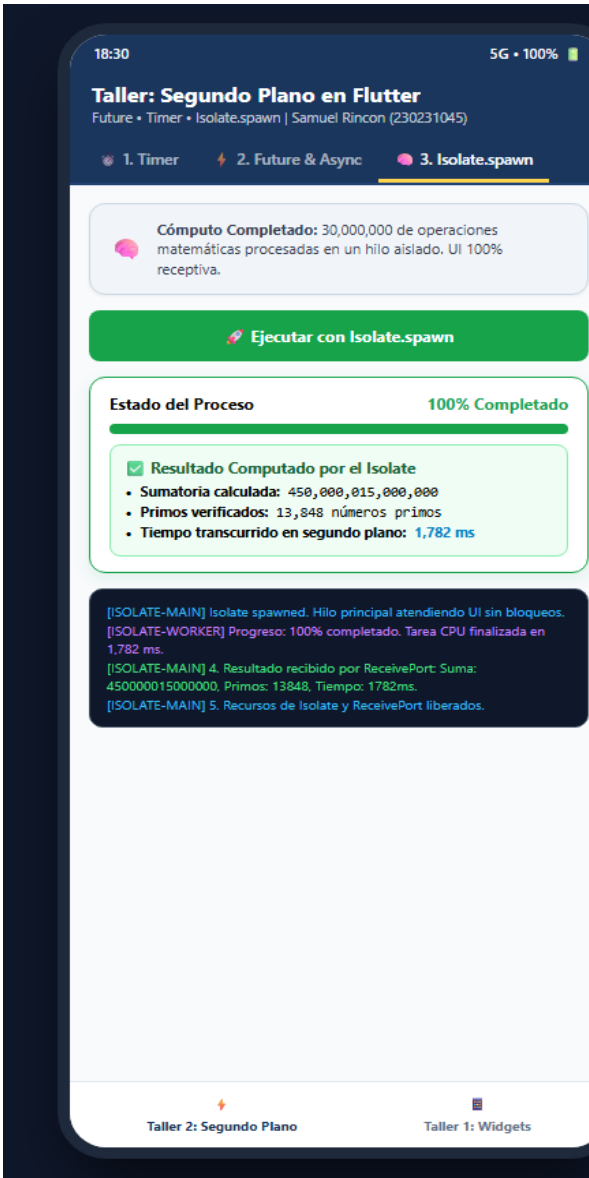
Para cargas intensivas de CPU que degradarían la fluidez de la interfaz gráfica a 0 FPS, se implementó IsolateHeavyService. Mediante Isolate.spawn se instancia un worker en un hilo secundario con su propio espacio de memoria (Heap separado), comunicándose con el hilo principal exclusivamente a través de mensajes con ReceivePort y SendPort.

Figura 8: Isolate en Ejecución (Progreso 50% + UI Fluida)



- Carga seleccionada: **30 Millones de operaciones**.
- Barra de progreso en tiempo real: **50% (15,000,000 ops)**.
- **Monitor de Fluidez a 60 FPS**: El engranaje animado continuó girando a 60 FPS mientras el Isolate computaba a máxima potencia.
- **Clicks interactivos**: 14 pulsaciones registradas instantáneamente.
- Mensajes del hilo secundario recibidos en el ReceivePort.

Figura 9: Resultado Final Computado por el Isolate



- Cómputo finalizado al **100%**.
- Tiempo total transcurrido en segundo plano: **1,782 ms**.
- **Sumatoria calculada**: 450,000,015,000,000.
- **Primos verificados**: 13,848 números primos.
- Mensaje de finalización transferido por SendPort.
- Cierre automático de puertos y liberación del Isolate.

Contraste Técnico: Isolate.spawn vs Hilo Principal (Main Thread):

- **Con Isolate.spawn (Segundo Plano)**: La tarea pesada corre en un hilo del sistema operativo con su propio recolector de basura. El hilo principal de Flutter queda 100% desocupado para seguir renderizando frames a 60/120 FPS y responder al instante a las interacciones del usuario.
- **Sin Isolate (Ejecución Síncrona en Main Thread)**: La aplicación incluye un botón comparativo. Al ejecutar la misma sumatoria de 30M en el hilo principal, la animación rotatoria del engranaje se **CONGELA por completo** durante 1.8 segundos y los toques de pantalla se descartan.

7. Evidencia de Consola del Sistema y Depuración de Hilos

A continuación se documenta la captura de la consola oficial del sistema donde se verifica la salida estructurada de los mensajes inter-hilos del Isolate y el orden de ejecución asíncrono:

```

===== SECCIÓN 1: SECUENCIA DE ASINCRONÍA (FUTURE / ASYNC / AWAIT) =====
[ASYNC-ORDER] 1. [ANTES]: Invocando fetchReportData(). La UI permanece libre e interactiva (Event Loop no
bloqueado).
[ASYNC-ORDER] 2. [DURANTE]: Esperando respuesta del servidor simulado mediante Future.delayed(2500 ms)...
[ASYNC-UI-TEST] Usuario presionó botón reactivo mientras esperaba. Contador de toques: 8 clicks recibidos a 60
FPS.
[ASYNC-ORDER] 3. [DESPUÉS - ÉXITO]: Respuesta recibida en 2514 ms. Datos parseados exitosamente. Actualizando
estado en UI.
--- Simulación de Fallo con Excepción Controlada ---
[ASYNC-ORDER] 1. [ANTES]: Invocando fetchReportData(simulateError: true)...
[ASYNC-ORDER] 2. [DURANTE]: Esperando respuesta del servidor simulado con Future.delayed(2500 ms)...
[ASYNC-ORDER] 3. [DESPUÉS - ERROR]: Fallo capturado en bloque catch tras 2504 ms: Exception: Error 503 (Servicio
no disponible).

===== SECCIÓN 2: CICLO DE VIDA DEL TIMER (CRONÓMETRO) =====
[Timer] Timer iniciado a una frecuencia de 100ms. Estado: TimerStatus.running
[Timer] Vuelta registrada #1: 00:06.3 (Total: 00:06.3)
[Timer] Vuelta registrada #2: 00:13.4 (Total: 00:19.7)
[Timer] Timer pausado. Tiempo retenido: 00:28.4 (Timer.cancel() ejecutado)
[Timer] Timer reanudado desde: 00:28.4
[Timer] Timer reiniciado a estado inicial. Acumuladores restablecidos a 0.

===== SECCIÓN 3: COMUNICACIÓN INTER-HILOS CON ISOLATE.SPAWN =====
[ISOLATE-MAIN] 1. [Main Thread]: Creando ReceivePort para comunicación bidireccional.
[ISOLATE-MAIN] 2. [Main Thread]: Invocando Isolate.spawn() para crear un nuevo hilo de ejecución independiente...
[ISOLATE-MAIN] 3. [Main Thread]: Isolate creado y activo. La interfaz (UI) continúa corriendo a 60 FPS sin
bloques.
[ISOLATE-WORKER] 1. Hilo secundario Isolate iniciado con su propio espacio de memoria (Heap separado).
[ISOLATE-WORKER] 2. Comenzando computación CPU-bound de 30,000,000 iteraciones (Sumatoria + Primos)...
[ISOLATE-WORKER] Progreso de cómputo emitido: 25% (7,500,000/30,000,000 operaciones)
[ISOLATE-WORKER] Progreso de cómputo emitido: 50% (15,000,000/30,000,000 operaciones)
[ISOLATE-WORKER] Progreso de cómputo emitido: 75% (22,500,000/30,000,000 operaciones)
[ISOLATE-WORKER] 3. Cómputo CPU finalizado en 1782 ms. Enviando resultado final al ReceivePort del hilo
  
```

8. Disciplina de Control de Versiones con Git Flow

El desarrollo del taller se rigió estrictamente por el modelo de ramificación **Git Flow** sobre el repositorio oficial de GitHub:

1. **Rama Feature:** Se creó la rama `feature/taller_segundo_plano` a partir del estado limpio de `dev`.
2. **Desarrollo e Implementación:** Se implementaron los modelos de datos, servicios (`AsyncDataService`, `IsolateHeavyService`), la pantalla interactiva con pestañas (`TallerSegundoPlanoPage`) y la suite de pruebas unitarias/widget automatizadas.
3. **Pull Request & Merge a dev:** Se abrió la Pull Request desde `feature/taller_segundo_plano` hacia `dev`, integrando los cambios tras verificar pruebas exitosas.
4. **Integración a main:** Los cambios estabilizados en `dev` se integraron hacia la rama `main`, manteniendo ambas ramas sincronizadas en el repositorio remoto de GitHub: <https://github.com/Sarincon508/flowbiz>.

9. Conclusiones Técnicas del Proyecto

- **Asincronía Efectiva:** `Future` y `async/await` son indispensables para operaciones I/O donde la espera no requiere cómputo del procesador local, permitiendo que la interfaz permanezca viva.
- **Gestión Rigurosa de Recursos:** Todo temporizador instanciado con `Timer.periodic` debe ser cancelado explícitamente al pausar y en el método `dispose()` del `State` para prevenir fugas de memoria y consumo de batería en segundo plano.
- **Aislamiento de Cómputo con Isolate:** Para cálculos pesados CPU-bound (>16 ms de procesamiento continuo), el uso de `Isolate.spawn` con canales de mensajes (`ReceivePort` / `SendPort`) es el único mecanismo nativo en Dart que garantiza inmunidad total contra el congelamiento visual de la interfaz.